

Plan de Branching - Cordillera Platform

Documento técnico correspondiente al Parcial 2 de Desarrollo Full Stack III (DSY1106).

1. Objetivo del documento

Este documento describe la estrategia de branching utilizada en el repositorio **Cordillera Platform**, correspondiente al caso Grupo Cordillera.

El objetivo es evidenciar cómo el equipo organizó el trabajo colaborativo usando Git y GitHub, manteniendo control de versiones, separación de responsabilidades, Pull Requests, merges y resolución de conflictos.

2. Repositorio del proyecto

Repositorio principal:

`https://github.com/Nachovn12/cordillera-platform-parcial-2`

El proyecto se trabaja como **monorepo**, incluyendo:

- Frontend React 19 + Vite + Nginx.
- BFF Gateway.
- Data Service.
- KPI Service.
- Report Service.
- Docker Compose.
- Documentación y evidencias.

3. Estrategia utilizada

La estrategia utilizada se basa en **Git Flow simplificado**, adaptado al contexto académico del Parcial 2.

Ramas principales:

Rama	Propósito
main	Rama estable del proyecto. Representa la versión lista para entrega o producción.
develop	Rama de integración. Recibe cambios desde las ramas feature mediante Pull Request.
feature/*	Ramas de desarrollo por componente o funcionalidad.
hotfix/*	Rama reservada para correcciones urgentes sobre main. No fue necesario usarla en esta etapa.

4. Ramas utilizadas

Evidencia obtenida con:

`git branch -a`

Resultado principal:

```
develop
feature/bff-gateway
feature/data-service
feature/docker-compose-integration
feature/kpi-service
feature/report-service
main
remotes/origin/develop
remotes/origin/feature/bff-gateway
remotes/origin/feature/data-service
remotes/origin/feature/kpi-service
remotes/origin/feature/report-service
remotes/origin/main
```

5. Rama de integración final Docker

Para la integración final se creó la rama:

```
feature/docker-compose-integration
```

Motivo:

- `develop` ya contenía cambios acumulados.
- Se necesitaba integrar Docker Compose sin ensuciar directamente `develop`.
- Se debían agregar Dockerfiles por servicio.
- Se debía validar frontend con Nginx.
- Se debía probar la arquitectura completa con MySQL Docker.
- Se debía preparar evidencia para presentación.

6. Flujo de trabajo aplicado

El flujo utilizado por el equipo fue:

```
main
^
develop
^
feature/nombre-componente
```

Proceso:

- Cada integrante trabajó en una rama `feature`.
- Los cambios se validaron localmente.
- Se creó Pull Request hacia `develop`.
- Se revisaron los cambios.
- Se hizo merge hacia `develop`.
- La integración final Docker se trabajó en `feature/docker-compose-integration`.
- Al finalizar, esta rama se integrará mediante Pull Request hacia `develop`.

7. Distribución por integrante

Integrante	Responsabilidad	Ramas / componentes
Ignacio Valeria	Frontend, Report Service, documentación, integración Docker	feature/frontend feature/report-service feature/docker-compose-integration
Benjamin Palma	BFF Gateway, KPI Service	feature/bff-gateway feature/kpi-service
Benjamin Flores	Data Service	feature/data-service

8. Evidencia de merges

Evidencia obtenida con:

```
git log --oneline --graph --decorate --all -n 30
```

Merges identificados:

```
34a3a5e Merge pull request #12 from Nachovn12/feature/frontend
f236574 Merge pull request #8 from Nachovn12/feature/bff-respuestas-degradadas
1875379 Merge pull request #11 from Nachovn12/feature/report-service-tests
```

Estos merges evidencian integración mediante Pull Request hacia develop.

9. Pull Requests relevantes

PR	Rama origen	Propósito
PR #8	feature/bff-respuestas-degradadas	Mejoras en BFF Gateway y respuestas degradadas.
PR #11	feature/report-service-tests	Pruebas unitarias obligatorias de Report Service.
PR #12	feature/frontend	Integración del frontend ejecutivo.

10. Gestión de conflictos

Durante la integración se detectaron situaciones de conflicto potencial o inconsistencia, especialmente por cambios acumulados en develop y ajustes simultáneos en frontend, BFF, documentación y Docker.

Ejemplo documentado:

```
Rama de trabajo : feature/docker-compose-integration
Rama base      : develop
Situación      : develop ya tenía cambios previos en frontend, BFF, properties
Decisión       : no trabajar directamente sobre develop para evitar
                ensuciar la rama de integración
Solución       : crear feature/docker-compose-integration y realizar
                la integración ahí
```

También se resolvieron inconsistencias de configuración:

- Puerto BFF original documentado como 8080.
- Puerto real del proyecto definido como 8081.
- Puertos 8080 y 8082 detectados como potencialmente ocupados en equipos del laboratorio.
- Se mantuvo BFF en 8081.

- Se actualizaron README, Docker Compose y documentación para evitar contradicciones.

11. Conflicto documentado de configuración

Elemento	Problema	Resolución
Puerto BFF	Existían referencias históricas a 8080, pero el proyecto real usa 8081.	Se actualizó documentación y configuración activa a 8081.
MySQL local / XAMPP	XAMPP podía ocupar 3306.	Se configuró MySQL Docker en host 3307 y contenedor 3306.
container_name en Docker Compose	Podía generar conflictos en otros computadores.	Se eliminaron los container_name para que Compose genere nombres automáticos.
React 18 vs React 19	Jira mencionaba React 18, pero el proyecto real usa React 19.	Se documentó React 19 + Vite de forma coherente con package.json.

12. Beneficios de la estrategia

La estrategia de branching permitió:

- Separar responsabilidades por componente.
- Evitar cambios directos en main.
- Mantener develop como rama de integración.
- Revisar cambios mediante Pull Request.
- Documentar merges relevantes.
- Integrar Docker sin romper el avance previo.
- Facilitar colaboración entre los tres integrantes.
- Mantener trazabilidad para la defensa oral.

13. Validación final de ramas

Comando usado:

```
git branch --show-current
```

Resultado:

```
feature/docker-compose-integration
```

Esto confirma que la integración final se realizó en una rama feature, siguiendo el flujo definido.

14. Validación del remoto

Comando usado:

```
git remote -v
```

Resultado:

```
origin https://github.com/Nachovn12/cordillera-platform-parcial-2.git (fetch)
origin https://github.com/Nachovn12/cordillera-platform-parcial-2.git (push)
```

15. Relación con la rúbrica

Este plan responde al indicador de branching del encargo grupal:

- Estrategia clara y organizada.
- Uso de ramas `main`, `develop` y `feature/*`.
- Evidencia de merges mediante Pull Requests.
- Resolución de conflictos documentada.
- Gestión colaborativa del equipo.

También apoya la defensa oral, donde se debe explicar cómo la estructura de ramas favoreció la colaboración y el control de versiones.

16. Conclusión

El equipo aplicó una estrategia de branching basada en **Git Flow simplificado**, manteniendo separación entre ramas estables, integración y desarrollo por funcionalidad.

La rama `feature/docker-compose-integration` permitió integrar Docker Compose, Nginx, MySQL Docker, BFF Gateway y microservicios sin comprometer directamente `develop`.

La evidencia de ramas, merges y decisiones de resolución demuestra control de versiones y colaboración organizada para el Parcial 2.